

# Integrating FPGA-Based Acceleration in Industrial Motion Control System

CLAUDIO RUBATTU <sup>1</sup>, ANTONIO LEDDA <sup>1</sup>, FRANCESCO RATTO <sup>2</sup>, CHAITANYA JUGADE <sup>3</sup>,  
DIP GOSWAMI <sup>3</sup> (Member, IEEE), AND FRANCESCA PALUMBO <sup>2</sup> (Senior Member, IEEE)

<sup>1</sup>University of Sassari, 07100 Sassari, Italy

<sup>2</sup>University of Cagliari, 09123 Cagliari, Italy

<sup>3</sup>Eindhoven University of Technology, 5612 AZ Eindhoven, The Netherlands

CORRESPONDING AUTHOR: CLAUDIO RUBATTU (e-mail: crubattu@uniss.it).

This work was supported by ECSEL Joint Undertaking in H2020 Project IMOCO4.E, under Grant 101007311. Moreover, this work takes advantage of knowledge on operating system customization in part by MYRTUS Project through the European Union under Grant 101135183.

**ABSTRACT** Manufacturing processes increasingly depend on advanced production machinery that must deliver high quality and large volumes. This applies to die-bonding machines as well that, especially at the time being after the years of shortage, need to meet very high standards of speed and accuracy. To achieve this, these devices are exploring the use of computer vision algorithms for automatic recognition of wafer positioning and die size. Nevertheless, these systems are typically managed by software-only solutions, which may fall short under stringent execution time requirements. A promising solution is the use of heterogeneous platforms, combining general-purpose processors with reconfigurable hardware. Such platforms offer the flexibility to handle both software tasks, which benefit from operating system support, and critical functions requiring hardware acceleration. This article presents a closed-loop implementation of a vision-based multisensor control system for an industrial application. The implementation exploits the capabilities of system on module technologies to provide flexible input/output and software execution coupled with computing acceleration for the vision algorithm on the reconfigurable field-programmable gate array (FPGA) fabric. The FPGA coprocessor has been designed leveraging the high-level synthesis technology and optimized on a dataset of 10 k realistic images to meet the industrial use case's performance, communication, and accuracy requirements. Moreover, the resulting accelerator performance and resource utilization demonstrate the possibility of reaching state-of-the-art metrics of handwritten hardware designs while allowing for higher abstraction and productivity of the design process.

**INDEX TERMS** Field-programmable gate arrays (FPGAs), high-level synthesis (HLS), industrial motion control systems, multiprocessor systems on chips (MPSoCs), systems on module (SoMs).

## I. INTRODUCTION

### A. CONTEXT

In the realm of Industry 4.0, where manufacturing phases increasingly depend on intelligent production machinery, a pertinent case study is the wafer stage of a die-bonding machine [1]. Recently, we have experienced a severe chip shortage [2], and although the situation has eased, its future trend remains uncertain. Thus, improving chip fabrication is essential, and more efficient cyber-physical systems (CPSs) can help achieve this goal. CPSs [3] have introduced numerous new application domains for advanced computation

platforms at the edge, including industrial applications [4]. Within this context, the H2020 project IMOCO4.E [5] combines and leverages novel sensory information, model-based approaches, and industrial Internet-of-Things philosophies [6]. Its objective is to enhance mechatronic systems by making them smarter, more configurable, and more reliable while simultaneously boosting their performance. An IMOCO4.E's use cases (UCs) focuses on improving the production rate of a modern die-bonding machine.

In this regard, Jugade et al. [7] demonstrated that employing a multisensor approach, common in CPSs, and shifting

advanced processing to the edge is strategically advantageous for enhancing the efficiency of industrial systems (like die-bonding machines). In that study, a multirate multisensor fusion algorithm was introduced to complement traditional systems, aiming to reduce operational errors that arise from inaccurate chip positioning. Typically, these systems utilize industrial sensors (such as linear encoders) to provide feedback on the motor positions that move the machinery, but not on the positions at points of interest, which differ due to external disturbances. To address this issue, in [7], fast but noisy encoder measurements have been merged with accurate, delayed, and slow vision measurements, resulting in high speed, accurate position estimates. This approach was demonstrated in a purely simulated environment and proved to improve the control system's closed-loop performance, at the condition that sufficient acceleration of the *vision algorithm* (VA) employed for the vision measurement is guaranteed.

A possible solution to accelerate the VA are the modern field-programmable gate array (FPGAs) platforms offering comprehensive systems on module (SoM) support. SoMs incorporate all the core components of an embedded processing system (PS) [e.g., processor cores, communication interfaces (I/F), memory blocks, and hardware (HW) accelerators] in a single small board, easing the integration to application-specific peripherals (e.g., cameras). Due to their features, SoMs are becoming a popular solution for a wide range of industrial applications that require the use of i) their heterogeneous HW components for flexible, yet accelerated, processing of different tasks and ii) their standard I/F for handling communication protocols [8]. Indeed, FPGAs-based platforms have been already utilized for implementing digital control in motion control systems and other Industry 4.0 application scenarios [9], [10], [11].

Regarding the target IMOCO4E die-positioning use, the feasibility of the acceleration through an FPGAs has then been presented in [12]. There, the VA has been ported to FPGAs through the use of high-level synthesis (HLS), paving the way toward the adoption of this kind of acceleration platform from software (SW)-oriented developers. Indeed, high-level HW design SW tools, such as the HLS tools, do not require deep HW expertise and enable shorter time-to-market, often at the expense of lower efficiency than specialized hardware description languages (HDLs). In addition, creating different versions of the same accelerator might, for example, require just a change of the HLS specifications in terms of previously defined parameters that impact conditional statements, avoiding lengthening design time due to implementation or manual modification of HW components.

## B. OBJECTIVES AND CONTRIBUTIONS

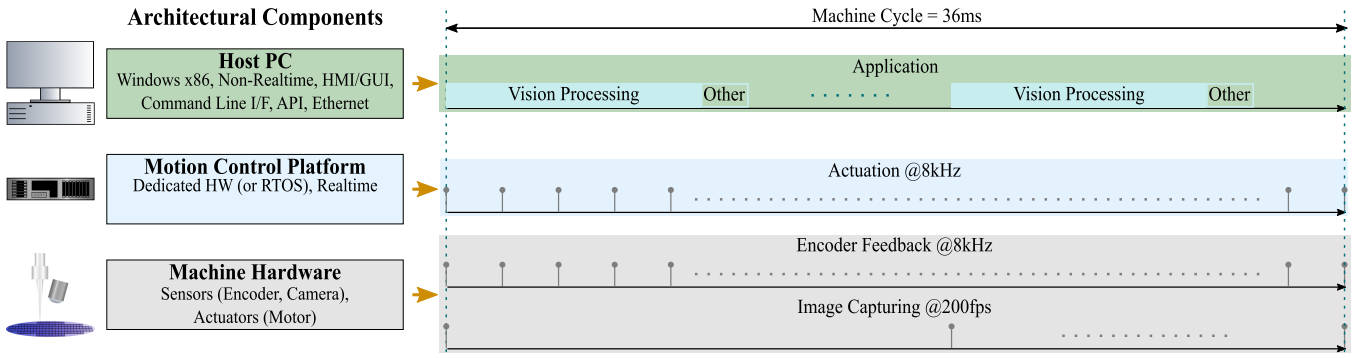
This work aims at demonstrating the advantages of the proposed multisensor solution ported on an FPGAs-based SoM, the AMD Kria K26 SoM, within a real-world industrial scenario. To fulfill this ambition, the some specific objectives have been identified.

- 1) The VA has to be tested and validated on a dataset on real images, as in the previous works only synthetic images were considered.
- 2) The coprocessor used to accelerate the VA has to be tuned with respect to dataset, as the algorithm presents some tunable parameters. The use of HLS design instead of tradition handwritten HW makes this process feasible and efficient for SW-oriented developers.
- 3) The full close-loop system has to be implemented to verify whether it meets the UC constraints. Such an implementation is made possible by the use of an advanced SoM that flexibly connects system's components.
- 4) Demonstrating that the use of HLS for the design of the accelerator allows to reach SoA performance and efficiency compared to handwritten HW solutions, which were not addressed in [12].

These objectives impose several advancements with respect to [7] and [12], which can be summarized in the following contributions.

- 1) An extensive dataset of 10 k images has been made available within project progress. This dataset captures typical conditions affecting die identification in realistic scenarios, such as the presence of nuances consequent to light and shadow effects, imperfectly straight edges resulting from the manufacturing process, and inclination of the wafer with respect to the conveyor belt axis.
- 2) The selection of the optimal coprocessor configuration capable of fulfilling the requirements demanded by the UC. The assessment and selection process exploited the availability of the dataset. A design-space exploration (DSE) aimed at exploring the tradeoff in terms of accuracy, execution times, and FPGAs resources has been carried out.
- 3) The integration of the proposed FPGAs-based implementation into the system architecture, which matches the selected UC of the H2020 project IMOCO4.E. SoM potentials are exploited by managing via SW the non-critical but time-constrained tasks that are the input acquisition from the camera and the output communication via ethernet. Moreover, proof that all the UC requirements, both in terms of computation and communication, are met is provided.
- 4) The positioning of the proposed VA accelerator with respect to the state of the art. A detailed comparison between the chosen HLS-based coprocessor configuration and other HDL-based solutions present in the literature, in terms of execution times and FPGAs usage, shows how the proposed implementation offers a viable SW-friendly alternative with respect to the state of the art.

The rest of this article is organized as follows. Section II outlines the background context of the UC (Section II-A) and the reference multirate multisensor system for industrial motion control [7] (Section II-B). Section III describes the setup related to the UC (Section III-A), the integration of the HW accelerator on the AMD Kria K26 SoM (Section III-B),



**FIGURE 1.** Overview of the initial UC architecture with execution times.

and the HLS-oriented implementation of a VA (Section III-C). Section IV presents the results, which include the following:

- 1) timing performance of the noncritical tasks executed onto the general-purpose processor of the AMD Kria K260 V vision starter kit (Section IV-C);
- 2) DSE exploring different configurations of the proposed HW accelerator with respect to a dataset of real input images (Section IV-A);
- 3) timing performance of the chosen coprocessor in comparison to those of an arm central processing units (CPUs) (included in the AMD Kria K26 SoM) and a commercial x86 processor, and comparison with state-of-the-art implementations based on specialized HDLs (Section IV-D);
- 4) HW resource usage related to the chosen VA accelerator and other system components necessary for managing the acceleration, and discussion with respect to the state-of-the-art solutions directly designed with HDLs (Section IV-E);
- 5) precision error in two different scenarios, each with varying parameters related to external disturbances and vision processing delays (Section IV-B).

Finally, Section V concludes this article.

## II. OVERVIEW OF THE INDUSTRIAL MOTION CONTROL

### A. USE CASE

As previously mentioned, this work is in the context of the European project IMOCO4.E [5], and in particular concerns the UC related to semiconductor manufacturing [7]. Fig. 1 (left part) shows the main components of the initial UC architecture from which this work started for the integration of the SoM into the closed-loop system. These components are briefly described as follows.

#### 1) HOST PERSONAL COMPUTER (PC)

A central host PC is essential for overseeing and coordinating the machine's operation. This host PC acts as the supervisory unit, responsible for initiating the machine's start-up and continuously monitoring the process flow and operational cycles. Key functions such as process control, feedforward control, vision processing, parameter setting, and tuning are all managed by this central platform. In addition, the host PC plays

**TABLE 1.** Overview of the UC Requirements

| Metric               | Requirement      |
|----------------------|------------------|
| Throughput           | 100 k units/hour |
| Machine's cycle time | 36 ms/unit       |
| VA time              | 1 ms             |
| Position error       | 6 $\mu$ m        |

a critical role in machine status monitoring, predictive maintenance, and diagnostics, ensuring the efficient and reliable operation of the machine.

#### 2) MOTION CONTROL PLATFORM

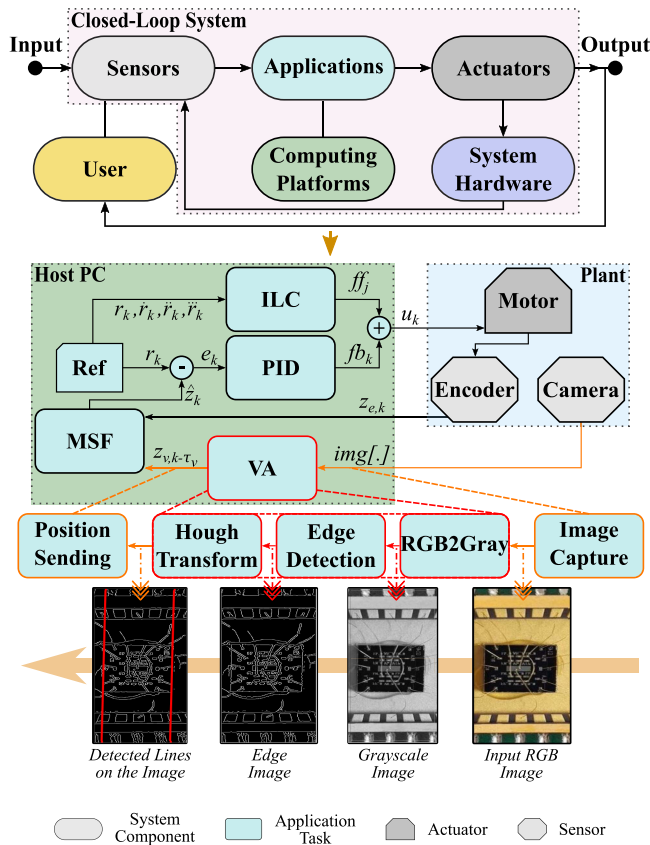
Motion control platforms are essential for providing accurate and predictable feedback control at high sampling rates, typically within the kHz range [13]. These platforms can operate in either a centralized, decentralized or hybrid manner, allowing for the real-time monitoring of component states. In addition, the motion control platform I/F with analog/digital input/output (I/O) and encoders, manages multiple servo axes in advanced equipment, and executes multiple-input multiple-output control.

#### 3) MACHINE HW

The machine HW consists of encoders, motors, physical components, and cameras. The I/O systems, motors, and encoders are handled by an embedded real-time motion controller. Encoders, which serve as sensors for tracking the position of the motor connected to the wafer table, can be sampled at different rates up to 20 kHz, depending on the desired precision for the application. Cameras, interfaced with the application processor, are focused on the wafer that contains an array of dies and should be selected to maintain a high frame rate for the image stream in order to optimize real-time performance.

#### 4) UC REQUIREMENTS

With respect to the UC requirements (see Table 1), these involve timing and accuracy metrics. Since the die-cutting machine selected for the UC must achieve a minimum throughput of 60 k units/hour, and potentially up to 100 k units/hour (equivalent to 36 ms/unit), timing requirements are directly



**FIGURE 2.** Integration of the multirate multisensor approach into the closed-loop system.

tied to the machine's cycle time. In this regard, Fig. 1 (right part) illustrates the task time sequence for each architectural components as expected during a machine cycle. The host PC is entrusted with all application tasks related to closed-loop system control. In this application, vision processing is the computationally most demanding task. Such a task executed as SW on general-purpose processors can take almost the entire time window required for a machine cycle (more than 30 ms [14]). Once the next positioning of the motors is calculated, the motion control platform provides the signals to drive the actuators. The machine HW consists of i) an encoder with a sampling frequency of 8 kHz (corresponding to a time period of 125  $\mu$ s), and ii) a camera with a high frame rate (e.g., 200 fps corresponding to a time period of 5 ms), a region of interest of  $88 \times 142$  pixels and an accuracy of 3.5  $\mu$ m/pixel. Nevertheless, accelerating the VA execution time to at least 1 ms can decrease the error in die position [7].

On the other hand, to ensure correct die cutting, the maximum position error must be less than the machine assembly precision, which is at maximum 6  $\mu$ m [15].

## B. MULTIRATE MULTISENSOR SYSTEM IN THE INDUSTRIAL MOTION CONTROL

Fig. 2 shows the block diagram of a closed-loop control system that includes multirate multisensor fusion, VA, and

control structure for industrial motion control application. All the details regarding the algorithms and the plant models are described in [7]. The application comprises various tasks (detailed in the following sections) and aims to improve the positioning accuracy of industrial motion control systems at different control points and points of interest by integrating a VA system with a linear encoder. This integration effectively reduces errors caused by external factors that may affect the precise positioning of dies, which a linear measurement system alone cannot detect. A key finding of [7] is that the closed-loop performance of the control system, specifically in terms of steady-state error (SSE) (i.e.,  $e_k$  at the end of each iteration), mean absolute error (MAE) (equal to  $\frac{1}{N} \sum_{k=0}^{N-1} |z_k - \hat{z}_k|$ ), and settling time (ST) (i.e., the time to achieve within 2% of the target position), worsens as the VA processing slows down. Therefore, minimizing vision delays by accelerating VA processing is crucial.

### 1) PROPORTIONAL-INTEGRATOR DERIVATIVE CONTROLLER

The discrete-time Proportional-Integrator Derivative (PID) controller equation is given as follows in order to favor clarity of analysis and achieve higher precision

$$fb_k = k_p \cdot e_k + k_i \sum_{m=0}^k e_m \cdot h_{\text{encoder}} + k_d \left( \frac{e_k - e_{k-1}}{h_{\text{encoder}}} \right) \quad (1)$$

where  $fb_k$  is the feedback control action,  $k_p$ ,  $k_i$ , and  $k_d$  are the proportional, integral, and derivative controller gains, respectively.  $h_{\text{encoder}}$  represents the sampling period of the linear encoder.  $e_k$  is the error signal

$$e_k = \hat{z}_k - r_k \quad (2)$$

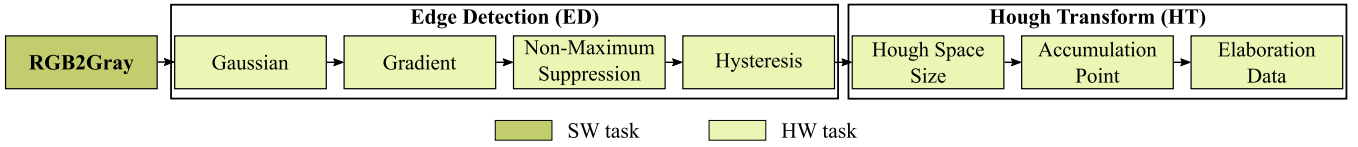
where  $r_k$  is the reference at the  $k$ th sampling instance, and  $\hat{z}$  is the estimation of the true die position obtained from the fusion of the two sensor data (coming from encoder and camera) [7].

### 2) FEEDFORWARD ITERATIVE LEARNING CONTROLLER

The Iterative Learning Controller (ILC) functions as a feedforward controller designed to enhance system performance across multiple cycles or iterations. It improves accuracy by learning from previous control errors and iteratively refining the control input to minimize these errors, thereby improving tracking precision and reducing cumulative error. The ILC is especially effective for repetitive tasks, such as pick-and-place operations in semiconductor manufacturing, where it adjusts and optimizes control inputs based on historical performance data. In Fig. 2,  $ff_j$  represents the feedforward signal, with  $j$  indicating the index of a specific repeating task operation. More in detail, its equation is the following:

$$ff_j = k_j \cdot \ddot{r}_j + k_a \cdot \dot{r}_j + k_v \cdot \dot{r}_j + k_C \cdot \text{sign}(\dot{r}_j) + k_s \cdot r_j + k_{s0} \quad (3)$$

where  $\ddot{r}$ ,  $\dot{r}$ , and  $r$  are the derivatives of the reference signal ( $r$ ),  $k_j$  is jerk,  $k_a$  is inertia,  $k_v$  is viscous friction,  $k_C$  is Coulomb friction,  $k_s$  is stiffness, and  $k_{s0}$  is spring offset coefficients to update the feedforward signal [16]. The combination of such



**FIGURE 3.** VA pipeline [7] with the proposed HLS-oriented implementation.

feedforward signal with the feedback control ( $fb_k$ ) sets the final control action ( $u_k = ff_j + fb_k$ ).

### 3) MULTIRATE SENSOR FUSION (MSF)

The multirate Kalman filter, which is the core result of [7], is employed to integrate data from two sensors: the linear encoder and the vision sensor. Typically, the linear encoder operates at a higher sampling rate, providing position feedback denoted as  $z_{e,k}$  at intervals of  $h_{\text{encoder}}$ . However, it does not account for external disturbances. On the other hand, a vision-based object detection algorithm, such as the one described below, processes camera images to ascertain the exact location of the object, including any disturbances, represented as  $z_{v,k}$ . Vision measurements are gathered at regular intervals of  $h_{\text{vision}}$  sample time, but image processing is computationally demanding. Consequently, the position information from the image captured at time-instance  $k$ , denoted as  $\text{img}[k]$ , becomes available after a delay of  $\tau_v$  samples. A multirate multisensor fusion algorithm integrates data from both the linear encoder and vision measurements to compute the estimated position information used in determining the control action. Specifically, the work in [7] shows that reducing the vision sensor's sampling period (e.g., at least to 1 ms with  $h_{\text{encoder}} = 125 \mu\text{s}$ ) decreases the error related to the die position.

### 4) VISION ALGORITHM

The VA is illustrated in Fig. 3. This task is designed to compute the distance of the chip from the left edge of the containment plane. It uses an acquisition sensor to capture an RGB image, which is then converted to grayscale. To enhance the chip edges, a detection algorithm based on the Canny edge detector operator [17] is utilized. Subsequently, the Hough transform (HT) [18] is applied to detect lines within the image, which can be adjusted to search for lines with specific orientations. In the given context, this approach leverages the fact that the camera views rectangular chips with linear edges. Furthermore, both the edge detection (ED) and HT processes were custom implemented for this UC. Specifically, the implementation focuses on identifying vertical edges in the input image and measuring their distance. Here are additional details on the implementation carried out in this article.

- 1) *RGB2Gray*: Since the conversion from RGB to grayscale depends on the resolution of the camera used for input acquisition, this step can be removed from the accelerated processing pipeline. Instead, it can be handled by an SW task that can adapt it to the resolution of the implemented accelerator, for example, by dividing

the input image into slices to entrust them to multiple instances of the accelerator.

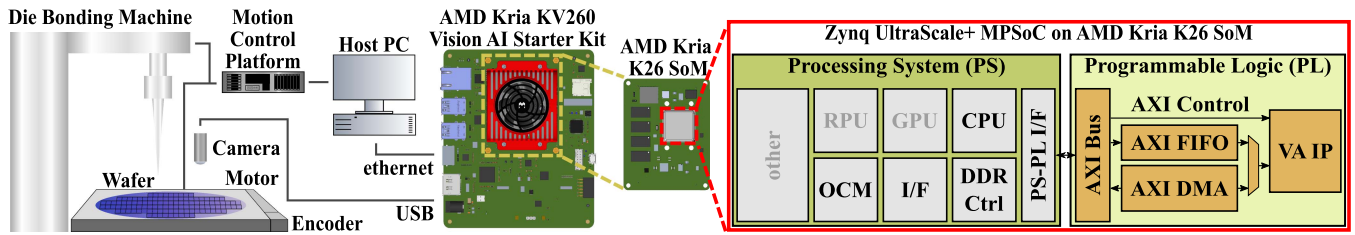
- 2) *ED*: The ED algorithm highlights the vertical edges of the image through the following four steps. First, the input image is convolved with a *Gaussian* (GA) filter to reduce noise from the environment where the image is captured. Next, the Sobel operator is applied to obtain two different gradients (GRs), task named *GR* in Fig. 3, representing both the magnitude and direction of the edges. The following step, *nonmaximum suppression* (NMS), calculates the gradient similarity to retain only the gradient with the highest magnitude value. Finally, the most significant edges are identified by applying a threshold function called *hysteresis* (H).
- 3) *HT*: This enables feature extraction of an image. Originally, such an algorithm was designed for detecting simple shapes like straight lines, circles, or ellipses, but was later extended to identify the positions of arbitrary shapes. In the context of the proposed VA, the HT is applied to find vertical lines with slopes limited to a small range of angles. The algorithm processing involves three steps. First, *Hough space size* (HSS) transposes the image resulting from the ED into Hough space. Next, in a task called *accumulation point* (AP), the number of lines that occur at each point in Hough space is examined, and if it exceeds a certain threshold, this identifies the position of vertical lines in a two-dimensional space called the Hough plane. Finally, the distances obtained in the Hough plane are converted into pixels corresponding to the identified lines, which can be plotted over the output image, and the distance to the edge of the image is calculated in the task named *elaboration data* (EL).

## III. TOWARD AN SOM-ORIENTED IMPLEMENTATION

Fig. 4 illustrates the proposed architecture for the UC of the H2020 IMOCO4.E project. This setup includes the following:

- 1) the die-bonding machine;
- 2) a silicon wafer transported by a conveyor belt, which integrates motor and linear encoder;
- 3) a motion control platform connected to the die-bonding machine and the conveyor belt;
- 4) a host PC used for handling the plant;
- 5) a camera for image acquisition;
- 6) an edge device involving a processor–coprocessor system.

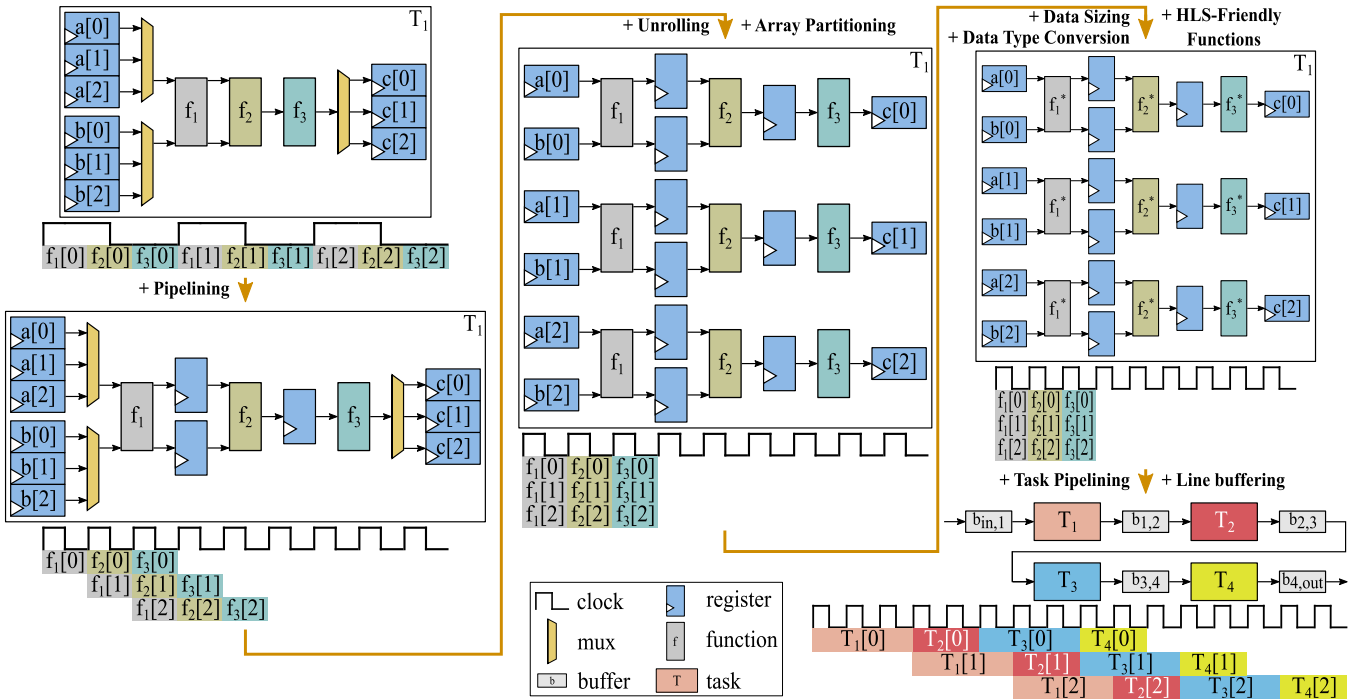
Differently from that described in Section II (and illustrated in Fig. 2), the host PC performs the application tasks



**FIGURE 4.** Overview of the envisioned implementation for the UC application.

except the ones associated with the VA, since the purpose is to separately accelerate their execution. For this reason, such tasks are handled by another computing platform, which can be placed at the edge (i.e., in close proximity to the plant). When data processing is required at the edge, the UC architecture leverages edge platforms, which are crucial for tasks like high-speed vision processing. High-speed VAs can be deployed on embedded edge devices—such as embedded graphics processing units (GPUs), FPGAs, and application specific integrated circuits (ASICs)—enabling applications in perception, localization, planning, and maintenance. These edge platforms can embed CPUs capable of running an operating system (OS), which enables orchestration of containerized applications and SW-based communication with a host PC. In addition, these platforms function as smart sensors for higher control layers, processing input image data with high-performance computing capabilities under strict time-sensitive constraints. In particular, such devices can integrate a reconfigurable substrate in order to exploit HW acceleration. Nevertheless, the use of FPGAs is still not common in closed-loop systems of industrial controls, in which applications are typically specified as an SW mapped onto general-purpose processors. Moreover, access from the FPGAs to the various I/F evaluates specific HW implementations, which can be arduous to SW developers. For these reasons, a potential solution that combines CPUs, FPGAs, and easy-to-use peripherals is represented by the SoMs based on heterogeneous multiprocessor systems on chip (MPSoCs) (such as the AMD Kria K26 SoM provided with the AMD Kria KV260 Vision AI Starter Kit [19]). In addition, critical tasks to be deployed into the FPGAs can be implemented by exploiting HLS, in order to favor a more SW-oriented processor-coprocessor approach. Compared to HDL solutions, HLS increases the level of abstraction used to define a specific functionality, which can lead to increased design productivity at the cost of a loss in performance [20]. In addition, HLS typically provides the intellectual properties (IPs) generation of the accelerator in order to easily integrate it into the processor-coprocessor system. This integration leverages specific I/F, for instance based on advanced eXtensible interface (AXI) protocols, which, in the case of HDL implementations, must be handled by the designer. HLS involves the compilation of high-level languages, such as SW programming languages, to FPGAs. This is crucial for the widespread adoption of FPGAs. HLS can also be understood as translating algorithmic descriptions into application-specific HW architectures. Most HLS tools

utilize C or C++ for algorithmic specification, though other languages such as Python [21], MATLAB [22], or SystemC are also used. Nevertheless, each HLS tool limits the design choices related to HW target (typically in terms of FPGAs vendors) and specification language. Examples of C-based commercial HLS tools include Vitis HLS [23] (former Vivado HLS), Stratus HLS [24], Catapult HLS [25], SmartHLS [26] (former LegUp). These tools facilitate the compilation of C, C++, and/or SystemC code for FPGAs or ASICs and can be effectively integrated with other frameworks to provide system-level feature support [27]. The resulting HW accelerator can be optimized and customized using directives that instruct the compiler on specific desired features. The syntax and semantics of these directives vary by tool. Common directives (or *#pragma*) include pipelining and unrolling, which enable the HW to execute loop iterations in a pipeline or in parallel, as well as memory resources, interface, and partitioning, which allow for the customization of memory architecture and access. In addition, commercial HLS tools generally provide support for performance estimation, design debugging, and verification. As an example of optimization through HLS, let consider a generic multitask streaming functionality such as the one described in Listing 1, where each task ( $T1$ ,  $T2$ ,  $T3$ , and  $T4$ ) is characterized by the iterative processing of specific functions (such as  $f1$ ,  $f2$ , and  $f3$ ). This functionality is structurally analogous to that to be accelerated (see Section II-B4). Typically, each task is optimized in order to decrease its impact in terms of either execution time or HW resources, or by choosing a balanced solution. Focusing on task  $T1$  without optimizations, this would present circuitry and execution time as shown in Fig. 5 (see the upper left portion). The addition of the *pragma pipeline* would lead to an increase in the number of registers to break the critical path, with a resulting gain in maximum frequency and interleaving of independent iterations within the task (see the lower left part of Fig. 5). In the case where the latency requirement is more stringent than the HW resource constraint, the use of unrolling would enable maximum parallelization of individual iterations only once the data are partitioned into independent memory blocks (see the central part of Fig. 5). Since the unrolling requires multiple instantiation of HW resources, its impact can be limited by minimizing the number of bits representing each data and the complexity of the involved functions (see the upper right portion of Fig. 5). Once individual tasks are optimized, execution time can be reduced by pipelining tasks, at the cost of including buffers



**FIGURE 5.** Overview of the main HLS optimization techniques for a streaming functionality (as described in Listing 1).

for passing data among tasks (see the lower right part of Fig. 5).

### A. UC-LEVEL IMPLEMENTATION

Regarding the integration of the processor–coprocessor system with respect to the architecture envisioned for the UC application, this required the implementation of SW functionalities for managing the acquisition of the input to be forwarded to the HW accelerator and the communication of the result of its processing to a host PC. The input images are captured using a camera connected to a USB port of the AMD Kria KV260 development board. These images can be then fed into the HW accelerator for processing. To handle the communication of the processed results from the HW accelerator, the implementation employs the HTTP protocol over ethernet. This strategy allows the processed data to be transmitted efficiently to a host PC. The host PC is responsible for further combining these results with other components within the closed-loop system, ensuring a cohesive and synchronized operation. This integrated approach not only enhances the performance of the system but also leverages the high-speed processing capabilities of the HW accelerator and the robust communication features of the HTTP protocol over Ethernet.

### B. SOM-LEVEL IMPLEMENTATION

The processor–coprocessor system is implemented on an AMD Kria K26 SoM, which embeds a Zynq UltraScale+ MPSoCs [19]. The presence of such an MPSoCs makes it easier to adhere to the processor–coprocessor paradigm [28]. The MPSoCs integrates a PS and programmable logic (PL)

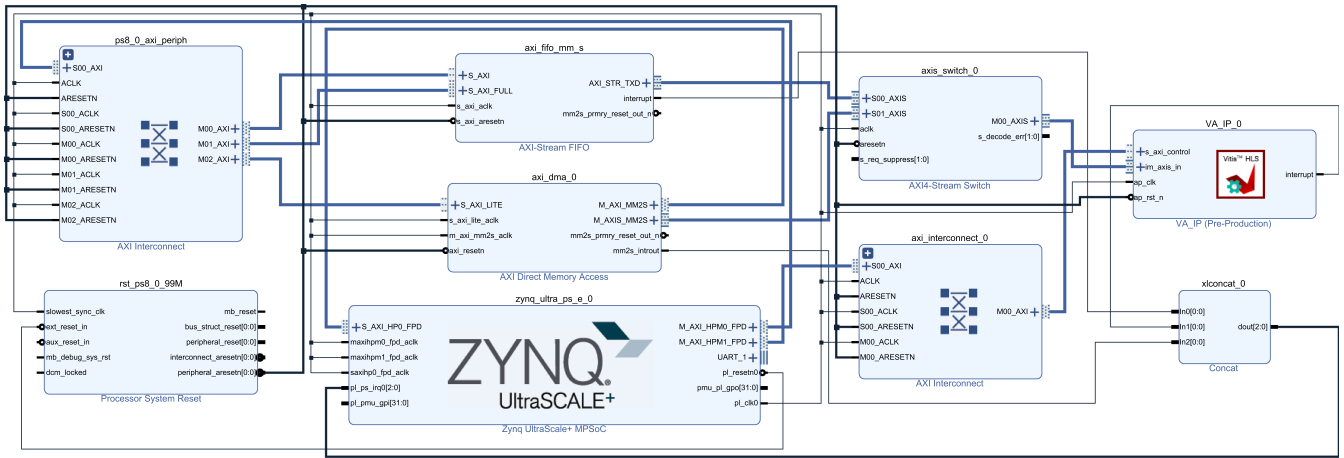
**LISTING 1:** HLS code snippet of a generic task-based streaming functionality.

```

1 void T1 (...) {
2     ...
3     for (i=0; i<3; i++) {
4         f1(a[i], b[i], x1, x2);
5         f2(x1, x2, y);
6         f3(y, c[i]);
7     }
8     ...
9 }
10 void T2 (...) { ... }
11 void T3 (...) { ... }
12 void T4 (...) { ... }
13
14 void top_module (...) {
15     ...
16     for (j=0; j<3; j++) {
17         T1 (...);
18         T2 (...);
19         T3 (...);
20         T4 (...);
21     }
22     ...
23 }

```

that communicate via dedicated I/F. The PS includes a quad-core CPUs (the sole processor used in this work), a dual-core real-time processing unit, a GPU, an on-chip memory, a DDR controller, various I/F such as Ethernet, and USB, several ports for the PS-PL I/F, internal interconnections, and additional modules such as a power manager and a safety and security module. Fig. 6 shows the block design as depicted on



**FIGURE 6.** SoM-level block design on Vivado design suite.

**TABLE 2. Use of Optimization Techniques in the HW Tasks**

| Optimization Technique |                                                               | HW Tasks        |                 |                     |                  |     |    |    |
|------------------------|---------------------------------------------------------------|-----------------|-----------------|---------------------|------------------|-----|----|----|
| Type                   | Description                                                   | GA <sup>P</sup> | GR <sup>P</sup> | NMS&HP <sup>P</sup> | HSS <sup>P</sup> | HSS | AP | EL |
| Pipelining             | reduces the interval for a loop to process new input data     | ✓               | ✓               | ✓                   | ✓                | ✓   | ✓  | ✓  |
| Loop Unrolling         | enables the parallel execution of independent loop iterations | ✓               | ✓               | ✓                   | ✓                | ✓   | ✓  | ✓  |
| Array Partitioning     | allows parallel memory access with smaller memory instances   | ✓               | ✓               | ✓                   | ✓                | ✓   | ✗  | ✗  |
| Data Sizing            | optimizes the resources actually needed for data processing   | ✓               | ✓               | ✓                   | ✓                | ✓   | ✓  | ✓  |
| Data Type Conversion   | decreases data handling resources without affecting precision | ✗               | ✗               | ✓                   | ✓                | ✓   | ✗  | ✓  |
| HLS-friendly Functions | optimize SW-oriented libraries for HW implementation          | ✓               | ✗               | ✓                   | ✓                | ✗   | ✗  | ✓  |
| Task Pipelining        | improves task-level concurrency and overall design throughput | ✓               | ✓               | ✓                   | ✓                | ✗   | ✗  | ✗  |
| Line Buffering         | minimizes the resources required to pass data between tasks   | ✓               | ✓               | ✓                   | ✓                | ✗   | ✗  | ✗  |

✓ and ✗ indicate the use or nonuse of the technique in the specific VA IPs task, respectively.

Vivado Design Suite. In this context, the PS is represented by a specific IPs (i.e., *Zynq UltraScale+ MPSoCs*). Except for this block associated with the PS, all the blocks in Fig. 6 are part of the PL and constitute the coprocessor. Such coprocessor has been implemented primarily consisting of the following:

- 1) an AXI-based bus for data and control communication between the processor and coprocessor (enabled with *AXI Interconnect* blocks);
- 2) a first-in first-out queues (FIFOs) for streaming data via the AXI-Stream protocol directly from the processor (*AXI-Stream FIFOs*);
- 3) a direct memory access (DMA) to facilitate AXI-Stream data transmission to the coprocessor while allowing the processor to continue executing other tasks (*AXI DMA*);
- 4) the core of the coprocessor, which is the HW accelerator for the VA (*VA IPs*).

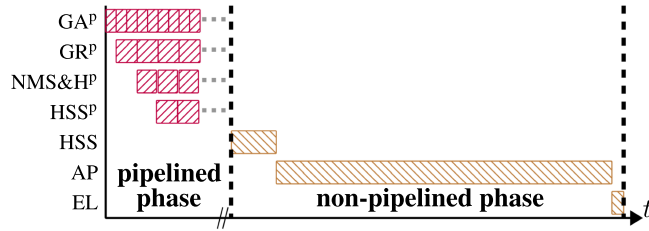
Note that although two modes of data transfer to the accelerator are available (via FIFOs and DMA) for flexibility reasons (FIFOs is faster for very small data because of the DMA overhead), the timing performance results (presented in Section IV) evaluate only the case where DMA is used.

### C. IP-LEVEL IMPLEMENTATION

The HW tasks of the VA (outlined in Section II-B4) have been implemented as an IPs block. Within the VA IPs, each HW task is defined as a submodule and optimized relative to its SW counterpart to comply with HLS guidelines.

However, for optimization purposes, the *NMS* and *H* tasks have been combined into a single block, while the *HSS* task has been split into two parts that can process either part or the entire input image (*HSS<sup>p</sup>* and *HSS*, respectively). Table 2 provides a summary of the optimization techniques applied to each submodule. Pipelined registers have been introduced in each loop within the submodules to enhance throughput with minimal additional area overhead. In addition, the choice to fully or partially unroll loops has decreased latency, though it has resulted in a significant increase in resource usage. Further timing optimizations involved using independent memory blocks to accelerate data access across nearly all tasks, with the exceptions being *AP* and *EL*. Conversely, resource constraints have been addressed by carefully managing the size and type of data for all registers and memories, and by converting floating-point data to fixed-point data where necessary (e.g., *NMS&H<sup>p</sup>*, *HSS<sup>p</sup>*, *HSS*, and *EL*). In addition, the computational complexity of certain mathematical functions required for processing (such as sine, cosine, arctangent, and square root) has been managed using optimized versions from HLS libraries or by employing tabulated values. To minimize execution time and reduce resource usage (particularly memory blocks), tasks that can be executed on independent sections of the image (including ED tasks and *HSS<sup>p</sup>*) are handled using pipelining and linear buffers.

Fig. 7 illustrates the execution phases of the VA IPs. The tasks in the pipeline process one pixel line of the input image



**FIGURE 7.** Task execution of the VA IPs.

at a time and are marked with  $P$ , while other tasks operate on the entire image and are executed only once per frame. Moreover, the amount of *HSS* and *AP* processing varies depending on the slope with respect to the vertical axis that can be evaluated for straight line identification. In particular, the range of angles has been defined through a configuration parameter called  $\theta$  expressed in degrees, which leads to different VA IPs. As an example, Listing 2 refers to a code snippet of the task *EL* for a specific  $\theta$  configuration, in which some optimization techniques are also applied.  $\theta$  parameter affects the implementation in terms of resource usage due to the dependence of the variable *val\_div*, which is equal to  $(\text{IMG\_HEIGHT}/2) * \sin(\theta)/\cos(\theta)$ . Nevertheless, since *val\_div* is required only for integer values of  $\theta$  between  $-\theta$  and  $+\theta$ , it can be defined as an array of size  $2\theta + 1$  of fixed-point integers (with 24 b in total, and 4 b for the integer field). This strategy requires the use of 2 optimization techniques: sampling the equation in terms more suitable for HLS (removing divisions and sine and cosine functions) and converting the data type (from float to *ap\_fixed*). The code snippet also gives examples of other HLS code optimization techniques: pipelining, unroll, and data sizing. Pipelining is applied via a *#pragma* directive, as is unrolling, which in this case was limited to a factor of 2 in order to not excessively increase the required FPGAs resources. On the other hand, the data sizes were optimized by reducing the number of bits required by integer variables, with particular attention to the range of values that can be represented (signed and unsigned).

#### IV. EXPERIMENTAL RESULTS

First, this section reports two DSEs. The former has been performed to understand how changing the  $\theta$  parameter in the VA IPs could affect in terms of accuracy, execution time, and FPGAs resources (see Section IV-A). Such a DSE enabled the identification of the most suitable configuration for the HW accelerator to be implemented with the complete FPGAs design flow. The latter has been required to verify the importance of the VA acceleration in the multisensor multirate approach (see Section IV-B).

In addition, this section shows how the fulfillment of the latency requirements given by the UC (see Section IV-C) is achieved. In particular, the evaluation regards the noncritical tasks specified as an SW and related to the following:

- 1) the acquisition of the input image by the AMD Kria K26 processor;

**LISTING 2:** HLS code snippet of the task *EL*, where pipelining, unrolling, data sizing, data type conversion and HLS-friendly functions are applied.

```

1  const ap_fixed<24,4,AP_RND> val_div[11] =
2  {-3.721949, 0.000000, -2.479412, 0.000000,
3  -1.239067, 0.000000, 1.239067, 0.000000,
4  2.479412, 0.000000, 3.721949};
5  ...
6  int16_t dataPeaks[MAX_PEAK_NUMBER]={0};
7  ap_fixed<24,4,AP_RND> theta;
8  ap_fixed<36,16,AP_RND> peak;
9
10 for(int8_t x=0;x<MAX_PEAK_NUMBER;x++) {
11 #pragma HLS UNROLL factor=2
12 #pragma HLS PIPELINE
13   theta=peakCoordinates[x+MAX_PEAK_NUMBER]
14     *0.01745;
15   int8_t the_int=(theta*100);
16   int8_t ind=the_int+5;
17   peak=peakCoordinates[x];
18   int16_t distance=peak-val_div[ind];
19
20   if(distance<0) distance=0;
21   else if(distance>(IMG_WIDTH-1))
22     distance=(IMG_WIDTH-1);
23
24   dataPeaks[x]=distance;
25 }
26 ...

```

- 2) its subsequent RGB-to-grayscale conversion to be forwarded toward the coprocessor;
- 3) the ethernet-based processor-to-host communication associated with the result of the VA accelerated in the coprocessor.

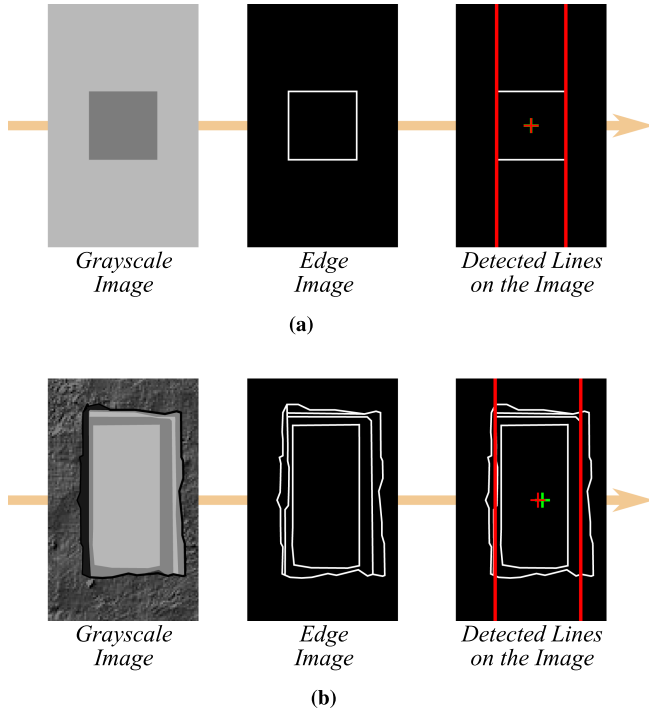
Finally, the implementation selected in Section IV-A has been compared with the state of the art in terms of execution time (see Section IV-D) and FPGAs resource utilization (see Section IV-E).

To evaluate the VA IP and its integration within a full processor-coprocessor environment, the Vitis HLS, and Vivado Design Suite (version 2023.1) have been utilized.

The coprocessor, incorporating the integrated VA IPs, was synthesized using the Vivado Design Suite at a maximum frequency of 230 MHz. Thus, all subsequent results are based on this reference frequency. In addition, the VA IPs has been tailored for the specific UC scenario, which involves processing input images of size  $88 \times 142$  pixels, with each pixel represented in grayscale format.

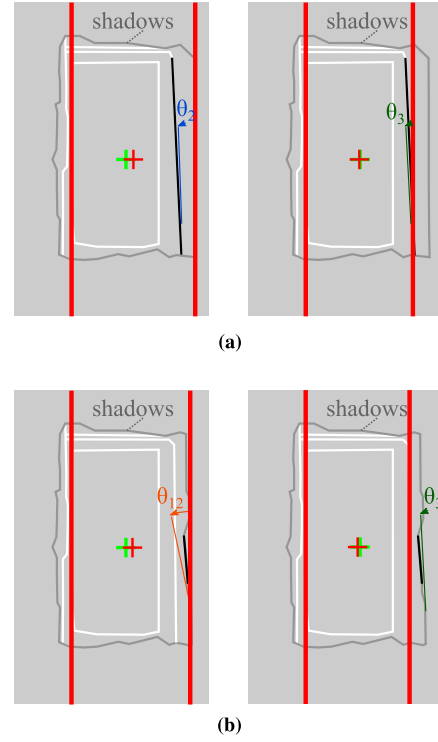
##### A. DSE: SYSTEM ON MODULE

In [7], the HW accelerator had been evaluated with a synthetic input image, where the chip was represented as a dark gray square centered in a light gray background [see Fig. 8(a)], no slope nor rotation to the vertical axis were considered. In this work, an assessment of the VA IPs over a dataset of 10k real images (still having a size of  $88 \times 142$  pixels) is performed to evaluate its performance and efficiency in identifying the



**FIGURE 8.** Die position identification with the synthetic input image used in [12] and a real image (representation) as in the dataset used in this work. (a) Synthetic input. (b) Real input.

position of the chip in the reference UC scenario. The real images provided as input have varying characteristics in terms of shape, inclination, nuances, and shadows. For example, in Fig. 8(b) a realistic image<sup>1</sup> affected by shadowing effect is shown. Fig. 8(a) and (b) shows, respectively, how the identification of the die center is perfectly accurate in the synthetic image case and how it may deviate from the correct value (labeled in the reference dataset) in the real image case. In this regard, investigating how the  $\theta$  parameter of the VA could impact accelerator processing turned out to be important. As already indicated in Section III-C, this parameter represents the range of angles with respect to the vertical axis in which the VA IPs is capable of identifying straight lines present in the input image. Evidently, when the chip has a slope beyond the  $\theta$ s range, identification may be more difficult if not incorrect. Fig. 9 depicts that, starting from the edge image, the identification of the die center may change due to the configuration of this HT parameter. In particular, wider ranges of  $\theta$  are affected by erroneous identifications due to more inclined dies [see Fig. 9(a)]. Similarly, also sloping edges associated with manufacturing imperfections or shadows may lead the HT algorithm to erroneous solutions [see Fig. 9(b)]. Increasing theta also implies additional HW resources and larger execution times, since it requires more computational complexity to examine the lines/edges with higher slopes. Specifically,  $\theta$  variations affect the processing of tasks *HSS*



**FIGURE 9.** Die position identification with different values of the  $\theta$  range parameter. (a) Inclined die edges. (b) Inclined shadows.

and *AP*, so that a value of  $\theta$ -dependent clock cycles,  $cc(\theta)$ , should be considered. The mathematical are reported hereafter

$$cc(\theta) = cc_{HSS}(\theta) + cc_{AP}(\theta) \quad (4)$$

$$cc_{HSS}(\theta) = \frac{N_{\theta}(\theta) \cdot N_{\rho}(\theta) \cdot IMG_{size, pad}}{uf_{HSS}} \quad (5)$$

$$cc_{AP}(\theta) = \frac{2 \cdot N_{\theta}(\theta) \cdot N_{\rho}(\theta)}{uf_{AP}} \quad (6)$$

$$N_{\rho}(\theta) = IMG_{width} \cdot \cos(\max(\theta)) \quad (7)$$

$$N_{\theta}(\theta) = 2 \cdot \theta + 1 \quad (8)$$

$$IMG_{size, pad} = (IMG_{width} - 6) \cdot (IMG_{height} - 6) \quad (9)$$

where  $\theta$  is expressed in degrees,  $N_{\theta}(\theta)$  and  $N_{\rho}(\theta)$  are the number of values considered in the Hough Space ( $\rho$  values for the y-axis and  $\theta$  values for the x-axis),  $uf_{HSS}$  and  $uf_{AP}$  are the unrolling factors used in the HLS pragmas of the tasks *HSS* and *AP*, and  $IMG_{size, pad}$ ,  $IMG_{width}$ , and  $IMG_{height}$  represent size (excluding padding), width and height of the input image, respectively. Please note that, given these relationships, the chosen unrolling factor affect not only the execution time but also the required HW resources.

All these functional and nonfunctional considerations motivate the need for implementing different versions of the VA IPs with varying  $\theta$  before any closed-loop implementation. It must be noted that having adopted an HLS-based design paradigm, the whole DSE process was facilitated since all the

<sup>1</sup>Real images cannot be included in the article due to nondisclosure agreements.

**TABLE 3.** Comparison Among Different  $\theta$ -Dependent Configurations of the Proposed VA IPs in Term of Estimation Error ( $\mu$  Mean Value,  $\sigma$  Standard Deviation, and  $\max abs$  Maximum in Absolute Value) Achieved With a Dataset of 10 k Real Input Images

| HW configuration<br>$\theta$ range[°] | Error [ $\mu\text{m}$ ] ([%]) |             |              | Execution<br>times [ $\mu\text{s}$ ] | PL resource usage [%] |     |      |     |
|---------------------------------------|-------------------------------|-------------|--------------|--------------------------------------|-----------------------|-----|------|-----|
|                                       | $\mu$                         | $\sigma$    | $\max abs$   |                                      | FF                    | LUT | BRAM | DSP |
| $\pm 2$                               | 2.02 (2.29)                   | 1.41 (1.60) | 8.17 (9.28)  | 35.67                                | 6                     | 20  | 0    | 3   |
| $\pm 3$                               | 1.79 (2.03)                   | 1.27 (1.44) | 5.49 (6.24)  | 37.98                                | 6                     | 22  | 0    | 3   |
| $\pm 4$                               | 1.98 (2.25)                   | 1.37 (1.56) | 7.48 (8.50)  | 43.54                                | 7                     | 24  | 0    | 3   |
| $\pm 5$                               | 1.98 (2.25)                   | 1.38 (1.57) | 9.35 (10.63) | 47.03                                | 7                     | 26  | 2    | 4   |
| $\pm 8$                               | 2.01 (2.28)                   | 1.57 (1.78) | 7.80 (8.86)  | 59.39                                | 8                     | 31  | 2    | 5   |
| $\pm 12$                              | 2.01 (2.28)                   | 1.57 (1.78) | 7.80 (8.86)  | 77.84                                | 9                     | 38  | 3    | 6   |

$\theta$  is a parameter corresponding to the range of angles representing the inclination of the chip in the input image evaluated for identification. Estimations extracted with Vitis HLS of execution times (@230MHz) and resources usage (related to the AMD Kria K26 SoM) for the VA IPs are also reported.

configurations under scrutiny were automatically generated from the different high-level descriptions by tuning  $\theta$ .

Table 3 highlights this tradeoff in selecting a specific VA IPs configuration. In particular, the best accuracy has been obtained with  $-3 \leq \theta \leq 3$ , since the maximum absolute error is the only one lower than the machine assembly precision ( $6 \mu\text{m}$ , see Section II-A). Although the average error is also reasonably small for other HW configurations, this is higher for  $-2 \leq \theta \leq 2$  (due to shadowing effect) and, starting from  $-4 \leq \theta \leq 4$  (due mostly to chip rotations effects), grows as  $\theta$  increases until saturation is reached. Regarding execution times, as expected, an increase with  $\theta$  is observed. Nonetheless, no critical values are to be reported since all the configurations allow to meet the timing requirement of 1 ms.

With respect to the HW resources, the percentage of usage on the target FPGAs remains limited to low values, except for the look-up tables (LUTs), which increase as  $\theta$  varies, reaching up to almost doubling from the smallest to the largest  $\theta$  (but increasing by only 2% in cases evaluated with low  $\theta$  s).

In summary, the best HW configuration corresponds to the one with  $-3 \leq \theta \leq 3$ . It is the most accurate, with a very small penalty in execution time (+6.5%) and FPGAs resource utilization (+10% of LUTs, corresponding to +2% with respect to the total) compared to the most efficient case, which is the one with  $-2 \leq \theta \leq 2$ . Therefore, this value of  $\theta$  has been chosen to implement the VA IPs embedded in the coprocessor, for which the results are presented in the following sections. Please note that, although in the considered scenario all the identified accuracy and processing requirements are met with a simple edge detector, from a purely algorithmic perspective, DSE results could improve with the application of different edge detectors. Their use could, in principle, lead to a more straightforward linear relationship between  $\theta$  and accuracy. Such algorithmic optimization though goes beyond the scope of this article.

## B. DSE: CLOSED-LOOP SYSTEM

To assess the effectiveness of the closed-loop control system that integrates the multirate multisensor fusion Section II-B3 and utilizes ED and HT Section II-B4 as a VA, a DSE across different parameters and scenarios has been conducted. This evaluation targets high-precision industrial motion control applications, such as pick-and-place operations in

**TABLE 4.** Closed-Loop Performance of MSF in the Scenario 1

| Scenario 1: $\Delta_1 = 20$ , $\Delta_2 = 15$ , $\Delta_3 = 10$ , $\Delta_4 = 5$ , and $\Delta_5 = 0$ . |                             |                       |              |       |       |       |        |  |
|---------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------|--------------|-------|-------|-------|--------|--|
| $h_{\text{encoder}}$<br>[ms]                                                                            | $h_{\text{vision}}$<br>[ms] |                       | Die position |       |       |       |        |  |
|                                                                                                         |                             |                       | 1            | 2     | 3     | 4     | 5      |  |
| 0.125                                                                                                   | 0.125                       | SSE [ $\mu\text{m}$ ] | 0.138        | 0.162 | 0.208 | 0.11  | 0.0312 |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 0.826        | 1.216 | 1.362 | 0.364 | 0.022  |  |
|                                                                                                         |                             | ST [ms]               | 25.32        | 29.43 | 24.38 | 27.44 | 29.51  |  |
| 0.125                                                                                                   | 0.25                        | SSE [ $\mu\text{m}$ ] | 0.200        | 0.280 | 0.326 | 0.191 | 0.0501 |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 1.164        | 1.620 | 1.836 | 0.232 | 0.035  |  |
|                                                                                                         |                             | ST [ms]               | 18.62        | 25.34 | 30.43 | 25.73 | 20.49  |  |
| 0.125                                                                                                   | 0.5                         | SSE [ $\mu\text{m}$ ] | 0.431        | 0.331 | 0.418 | 0.112 | 0.151  |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 2.724        | 2.913 | 2.663 | 1.960 | 0.086  |  |
|                                                                                                         |                             | ST [ms]               | 19.32        | 23.52 | 30.11 | 28.12 | 23.16  |  |

All the deviations ( $\Delta_i$ ), where  $i = 1, 2, 3, 4, 5$  for 5 dies are in  $\mu\text{m}$ .

**TABLE 5.** Closed-Loop Performance of MSF in the Scenario 2

| Scenario 2: $\Delta_1 = 40$ , $\Delta_2 = 20$ , $\Delta_3 = 10$ , $\Delta_4 = 5$ , and $\Delta_5 = 0$ . |                             |                       |              |       |       |       |       |  |
|---------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------|--------------|-------|-------|-------|-------|--|
| $h_{\text{encoder}}$<br>[ms]                                                                            | $h_{\text{vision}}$<br>[ms] | m                     | Die position |       |       |       |       |  |
|                                                                                                         |                             |                       | 1            | 2     | 3     | 4     | 5     |  |
| 0.125                                                                                                   | 0.125                       | SSE [ $\mu\text{m}$ ] | 0.246        | 0.283 | 0.220 | 0.186 | 0.065 |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 0.932        | 1.463 | 1.530 | 0.364 | 0.030 |  |
|                                                                                                         |                             | ST [ms]               | 27.11        | 30.43 | 26.72 | 29.31 | 31.47 |  |
| 0.125                                                                                                   | 0.25                        | SSE [ $\mu\text{m}$ ] | 0.259        | 1.910 | 0.511 | 0.231 | 0.091 |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 1.466        | 1.814 | 2.313 | 1.112 | 0.070 |  |
|                                                                                                         |                             | ST [ms]               | 19.47        | 32.40 | 33.61 | 28.60 | 23.73 |  |
| 0.125                                                                                                   | 0.5                         | SSE [ $\mu\text{m}$ ] | 0.611        | 0.530 | 0.502 | 0.311 | 0.220 |  |
|                                                                                                         |                             | MAE [ $\mu\text{m}$ ] | 3.180        | 3.412 | 3.120 | 1.170 | 0.090 |  |
|                                                                                                         |                             | ST [ms]               | 24.30        | 27.33 | 20.01 | 33.28 | 29.61 |  |

All the deviations ( $\Delta_i$ ), where  $i = 1, 2, 3, 4, 5$  for 5 dies are in  $\mu\text{m}$ .

semiconductor manufacturing. As detailed in [7], Tables 4 and 5 illustrate the performance of the closed-loop control system in terms of SSE, MAE, and ST.

In the pick-and-place task, ideally, five semiconductor dies are sequentially positioned with consistent spacing. The camera's region of interest is centered on the first die at  $z_0 = 0 \mu\text{m}$ . The objective is to align the first die with a target position of  $220 \mu\text{m}$  by the end of the first iteration. In a different scenario where external disturbances impact the die position, the linear encoder sensor, which only monitors the position

**TABLE 6. Execution Time Evaluation ( $\mu$  Mean Value,  $\sigma$  Standard Deviation,  $Min$  Minimum Value, and  $Max$  Maximum Value) of the SW Tasks**

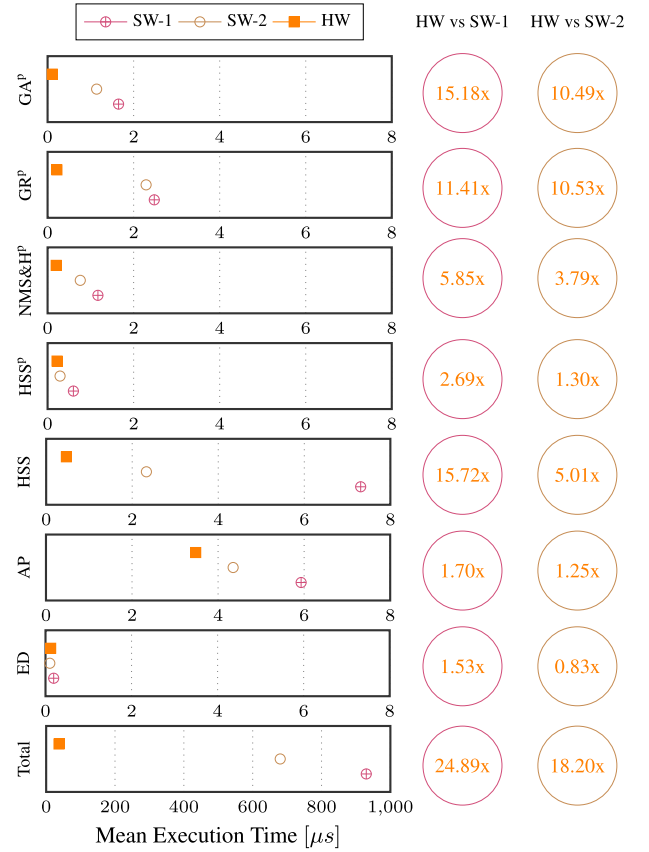
| Task             | Execution times [ $\mu s$ ] |          |        |        |
|------------------|-----------------------------|----------|--------|--------|
|                  | SW-1 (on Arm Cortex-A53)    |          |        |        |
|                  | $\mu$                       | $\sigma$ | min    | max    |
| Image capture    | 280.00                      | 13.58    | 260.00 | 400.00 |
| RGB2Gray         | 110.00                      | 5.80     | 100.00 | 170.00 |
| Position sending | 80.00                       | 26.77    | 50.00  | 250.00 |

of the wafer table, cannot detect the inaccuracies. However, the vision sensor, utilizing captured images and a VA, can precisely determine the actual position of the die.

Tables 4 and 5 illustrate two different scenarios. In the first scenario, an initial disturbance of  $\Delta = 20 \mu m$  is applied to the first semiconductor die, followed by disturbances to dies two, three, and four, with each subsequent disturbance decreasing by  $5 \mu m$ . The fifth semiconductor die remains unaffected. In the second scenario, an initial disturbance of  $\Delta = 40 \mu m$  is applied to the first die, and disturbances to dies two, three, and four decrease by a factor of 2. The fifth die experiences no disturbance. In these scenarios, the linear encoder operates with a sampling period of  $h_{\text{encoder}} = 0.125$  ms, while the VA is tested with sampling periods of  $h_{\text{vision}} = 0.125$  ms,  $0.25$  ms,  $0.5$  ms. The performance results for both scenarios are shown in Tables 4 and 5. Generally, the SSE values are below  $1 \mu m$  in most cases, though they vary depending on the disturbances. Performance metrics such as SSE, MAE, and ST tend to worsen when  $h_{\text{vision}} = 0.5$  ms compared to the case with  $h_{\text{vision}} = 0.125$  ms. Thus, minimizing vision delay and using a shorter  $h_{\text{vision}}$  is advantageous, and the proposed accelerator's execution time can accommodate these sampling requirements.

### C. PROCESSOR EXECUTION TIME EVALUATION

Since the UC application is composed of sequential SW tasks without any parallelism, its response time is the sum of the execution times of each task, which represents its critical path [29]. Table 6 summarizes latency contributions of the SW tasks executed  $10k$  times and mapped onto the PS of the Kria K26 (SW-1), which features a quad-core Arm Cortex-A53 MPCore running up to  $1.5$  GHz. Results show how the total execution time (also called response time) meets the requirements mentioned in Section II-A4 also with the addition of the accelerator latency (reported in Section IV-A). Note that these run times are based on the SW implementation only. Nevertheless, a dependency will then be placed on the frame-per-second capability of the camera employed. Effectively, the minimum sampling period for the acquisition of a new frame (e.g.,  $5$  ms with  $200$  fps) will need to be added. Therefore, in order to meet the requirement of maximum sampling every ms, high-speed cameras need to be employed [30]. On the other hand, the latency results also disregard the time required to transfer the output of the coprocessor in the local network (from the Kria KV260 to the host PC), which is in the order of a tenth of a millisecond in the implemented setup.

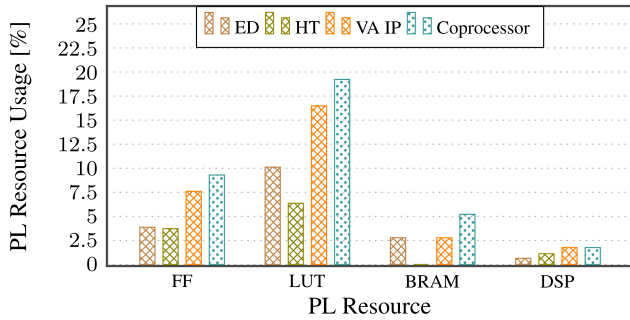


**FIGURE 10. Comparison among hardware (HW) and SW versions of the VA IPs in terms of mean execution time (on the left) and HW speed-up compared to the SW-1 and SW-2 values (on the right).**

The combined use of SW tasks (running on the CPUs) and HW (implemented in FPGAs) requires data transfer from the PS to the PL of the MPSoCs. In the specific UC, this transfer is influenced by the drivers used to manage the AXI DMA. The execution times for this transfer have been measured, with maximum values of  $81 \mu s$  for the configuration and  $69 \mu s$  for transferring the entire image to the coprocessor. Nevertheless, the configuration only needs to be set once during device initialization (rather than being managed for each frame), and the data transfer time to the accelerator is partially overlapped with the processing time related to the VA IPs, since the processing is pipelined by handling one line of the image at a time. Specifically, it does not overlap only with the tasks not involved in the pipeline (HSS, AP, and EL), which in aggregate have an execution time of  $4.07 \mu s$  out of the total  $37.37 \mu s$ . For these reasons, the maximum execution time for data transfer via AXI DMA can be effectively considered equal to  $35.7 \mu s$ . Finally, the transfer time for reading the output from the VA IPs register is almost negligible (in the order of microseconds).

### D. COPROCESSOR EXECUTION TIME EVALUATION

Fig. 10 illustrates a comparison between SW and HW versions in terms of mean execution time. The SW results were



**FIGURE 11.** PL resource usage of ED, HT, VA IPs, and the whole coprocessor (at 230 MHz) in terms of FF, LUTs, BRAMs, and DSP of the AMD Kria K26 SoM.

obtained from 10k runs of the VA SW IPs performed on: i) the PS of the Kria K26 (SW-1, as done in Section IV-C); and ii) a 6-core AMD Ryzen 5 5600 processor, which operates up to 4.4 GHz (SW-2). For a fair comparison, the execution of the VA was conducted using a single core for the SW cases (SW-1 and SW-2) and one accelerator for the HW case, with both being assessed at their maximum operating frequencies. All processing steps involved in the ED and the HT, as detailed in Section II-B4, were accelerated, leading to an overall speed-up of 24.89 $\times$  and 18.20 $\times$  compared to SW-1 and SW-2, respectively. This significant acceleration meets the constraints outlined in Section II-A4 and [7] for the VA to effectively support the Multirate Sensor Fusion (MSF) of the industrial motion control system depicted in Fig. 2, thereby enabling scenarios with improved precision error (see Section IV-B). In contrast, the SW executions evaluated (SW-1 and SW-2) may not fulfill the required time constraints (since their maximum values are 1.02 ms and 1.28 ms, respectively). This discrepancy arises because the speedup values are disproportionately favorable toward the proposed HW implementation, particularly for tasks that are executed multiple times per frame (i.e., pipeline tasks). Although higher operating frequencies of CPUs can result in speedups closer to 1 (or even lower in one case, i.e., for the ED task) for tasks that are more characteristic of general-purpose implementations. Nonetheless, the designed VA IPs completes its tasks in 37.37  $\mu$ s, well below the required 1 ms, even when accounting for processor–coprocessor communication overhead, including the latency associated with the DMA transferring the image to the coprocessing unit. In fact, thanks to the acceleration to 37  $\mu$ s of the set of VA HW tasks, and by summing the measured maximum execution times of the SW tasks shown in Table 6 (which results in 820  $\mu$ s) and the combined tasks related to the I/O data transfer (which is about 36  $\mu$ s), the requirement reported in Table 1 remains satisfied (since 893  $\mu$ s < 1 ms).

#### E. COPROCESSOR RESOURCE USAGE EVALUATION

Regarding resource usage, the coprocessor utilizes a small portion of PL resources, primarily allocated to the VA IPs (refer to Fig. 11), although it has been designed through HLS.

Resources required for VA IPs submodules associated with ED and HT are also reported. The results reflect the optimizations applied to the original SW code (as outlined in Table 2). Specifically, the 16.48% usage of LUTs is largely due to loop unrolling, which increases the number of HW blocks needed for parallel processing. In addition, the extensive use of the pipeline directive contributes to the 7.60% utilization of flip flop (FF) registers. Even though the accelerator processes the input image one pixel line at a time, it requires 4 blocks of 36 kb random access memory, which is 2.78% of the total block random access memories (BRAMs) available on the target FPGAs. The need for only 1.76% of digital signal processors (DSPs) resources is attributed to typical image processing operations in the tasks *GR* and *HSS*. Besides the resources for VA IPs, the coprocessor contains additional resources are used for DMA, FIFOs, and components managing clock, reset, and interconnect functions (for more details see Fig. 6). Despite this, the VA IPs allows for the integration of additional functionalities on the coprocessing unit (such as the MSF discussed in Section II-B3) or for the deployment of multiple instances of the VA IPs. This last case represents a scenario in which multiple accelerators can be deployed in case i) part of the proposed VA IPs is to be reused, or ii) the same VA IPs is to be used to process larger inputs without implying additional execution time than those presented in Section IV-D.

#### F. HLS-BASED COPROCESSOR VERSUS HANDWRITTEN HDL-BASED SOLUTIONS

This section proposes a comparison between the HLS-based implementation of the accelerator presented and solutions in the literature based on handwritten HDL. In particular, the comparison regards the two state-of-the-art algorithms (i.e., ED and HT), which are combined in the VA IPs. To the authors' knowledge, no other works using the same HLS tool and SoM can be involved for a direct comparison. Therefore, the proposed comparison focuses on the same type of adaptive technology, the FPGAs, but includes different vendors' platforms, such as AMD (formerly Xilinx) and Intel (formerly Altera), in the implementation of handwritten HDL solutions.

In terms of content, Tables 7 and 8 extend those presented in survey papers [31] and [32], which were, respectively, related to ED and HT. In addition, to allow a more fair comparison of implementations on FPGAs-based platforms belonging to different device families and produced by different vendors, two metrics have been used for normalization of FPGAs execution time and resource usage: time per pixel (ns/pixel) and resources per pixel (slices/pixel). The normalized execution time (ns/pixel) has already been used to directly compare efficiency relative to image size in [31] and [32]. Whereas, we are introducing the normalized resource usage, which is represented in terms of slices. The conversion in terms slices starting from a memory block can be derived by each LUTs with the following formula  $2^{in}$ , where  $in$  is the LUTs input size (i.e., 6-input LUTs corresponds to  $2^6 = 64$  b memory block), and by the knowledge that each slice contains a different

**TABLE 7. Comparison of Execution Times and FPGA Resources Between the Proposed Submodule for the ED and the Related State-of-the-Art Implementations**

| Metrics                            | [34]           | [35]            | [36]            | [37]            | [31]                    | This Work    |
|------------------------------------|----------------|-----------------|-----------------|-----------------|-------------------------|--------------|
| Image Size [pixel]                 | 360 × 280      | 512 × 512       | 512 × 512       | 512 × 512       | 512 × 512               | 88 × 142     |
| Frequency [MHz]                    | 27             | 292             | 250             | 272             | 56.7                    | 230          |
| Execution Time [ms/frame]          | 2.50           | 0.57            | 0.15            | 0.13            | 1.23                    | 31.18E-3     |
| Norm. Execution Time [ns/pixel]    | 24.80          | 2.17            | 0.57            | 0.50            | 4.69                    | 2.50         |
| FPGA-based Platform                | Altera Cyclone | Xilinx Virtex-5 | Xilinx Virtex-5 | Xilinx Virtex-5 | Altera/Intel Cyclone IV | AMD Kria K26 |
| LUT per Slice [unit/slice]         | 1              | 8               | 8               | 8               | 1                       | 8            |
| LUT Input size [unit]              | 4              | 6               | 6               | 6               | 4                       | 6            |
| Slices [slice]                     | -              | 4,553           | 23,904          | 17,928          | 3303                    | 20,925       |
| Memory Blocks [kB]                 | -              | 192             | 16,184          | 278             | 261                     | 18           |
| Total Used Resources [slice]       | -              | 7,553           | 276,779         | 22,272          | 133,803                 | 21,206       |
| Norm. Resource Usage [slice/pixel] | -              | 0.03            | 1.06            | 0.08            | 0.51                    | 1.70         |

**TABLE 8. Comparison of Execution Times and FPGA Resources Between the Proposed Submodule for the HT and the Related State-of-the-Art Implementations**

| Metrics                            | [38]              | [33]              | [39]              | [40]              | [32]         | This Work    |
|------------------------------------|-------------------|-------------------|-------------------|-------------------|--------------|--------------|
| Image Size [pixel]                 | 1024 × 768        | 1024 × 768        | 752 × 320         | 1000 × 1000       | 640 × 480    | 88 × 142     |
| Frequency [MHz]                    | 200               | 200               | 40                | 260               | 200          | 230          |
| Execution Time [ms/frame]          | 15.59             | 5.4               | 13.92             | 3.86              | 1.47         | 35.56E-3     |
| Norm. Execution Time [ns/pixel]    | 19.80             | 6.8               | 57.84             | 3.86              | 4.78         | 2.85         |
| FPGA-based Platform                | Altera Cyclone IV | Altera Stratix IV | Xilinx Spartan 3A | Xilinx Virtex VII | AMD Virtex-5 | AMD Kria K26 |
| LUT per Slice [unit/slice]         | 1                 | 1                 | 2                 | 8                 | 8            | 8            |
| LUT Input Size [unit]              | 4                 | 7                 | 4                 | 6                 | 6            | 6            |
| Slices [slice]                     | 29,431            | 4,115             | 3,692             | 82,673            | 1,119        | 16,196       |
| Memory Block [kB]                  | 381.5             | 200.5             | 105               | 828               | 203.1        | 0            |
| Total Used Resources [slice]       | 220,181           | 16,646            | 29,942            | 95,611            | 4,292        | 16,196       |
| Norm. Resource Usage [slice/pixel] | 0.28              | 0.02              | 0.12              | 0.10              | 0.01         | 1.30         |

number of LUTs ( $lut_s$ ), e.g., the AMD Kria K26 contains 8 6-input LUTs, while the implementation in [33] is adopting a 7-input LUTs. To the slices already considered in [31] and [32] ( $s$ ), those derived from the corresponding memory blocks expressed in bits ( $b_{mem}$ ) have been added to obtain the total used resources ( $res_{tot}$ ) using the following formula:

$$res_{tot} = s + \left( \frac{b_{mem}}{2^{in} \cdot lut_s} \right). \quad (10)$$

As an example, considering the total resources of this work given in Table 7, (10) becomes  $res_{tot} = 20,925[slice] + \left( \frac{18 \cdot 1000 \cdot 8[unit]}{2^6 \cdot 8[unit/slice]} \right) = 21,206[slice]$ .

Numerically speaking, in terms of normalized execution time, the proposed implementation of the ED achieves 2.50 ns/pixel, which is more efficient than [34] (24.80 ns/pixel) and [31] (4.69 ns/pixel), and closely aligns with the performance of [35] (2.17 ns/pixel). However, it falls short compared to the highly efficient methods of [36] (0.57 ns/pixel) and [37] (0.50 ns/pixel). With respect to the HT, the proposed solution achieves a normalized execution time of 2.85 ns/pixel, which is the most efficient among all methods. In this regard, it should be highlighted that the implementation involved in the VA IPs is limited to the detection of straight lines with a small slope from the vertical axis ( $-3$  to  $+3$  degrees), while the other approaches in the

literature include more extensive functionalities. In summary, with respect to the normalized execution time, the proposed HLS-based implementation of the ED and the HT are aligned with the handwritten HDL ones present in the state of the art, proving the suitability of the chosen approach for possible real-time applications where processing speed is critical.

Regarding the FPGAs resources, the proposed ED implementation utilizes the AMD Kria K26 FPGAs, with 20,925 slices and 18 kB of memory blocks. This configuration seems more resource-efficient in terms of memory usage compared to [35] (192 kB), [36] (16,184 kB), [37] (278 kB), and [31] (261 kB). Such efficiency related to the memory blocks is achieved at the cost of a superior usage of slices, but still in line with the works oriented to reduce memory resources: [36] (23,904 slices) and [37] (17,928 slices). For the HT implementations, the proposed work utilizes 16,196 slices and no memory blocks of the AMD Kria K26 FPGAs. As mentioned above, since the proposed implementation is more limited in terms of functionalities, it does not require the use of memory blocks, making it extremely memory-efficient compared to all other methods: [38] (381.5 kB), [33] (200.5 kB), [39] (105 kB), [40] (828 kB), and [32] (203.1 kB). Nevertheless, the absence of memory blocks led to an increment of slice usage. In particular, focusing on the normalized resource usage, all the handwritten HDL solutions achieve

better efficiency. In summary, with respect to normalized resource usage, the proposed implementations for both the ED and the HT fall shorter in terms of resource usage with respect to the more optimized handwritten HDL ones. Overall, the resource efficiency result was expected. HLS-based solutions generally improve design productivity at the cost of reduced efficiency. Nonetheless, in this particular case, it does not affect the final result, since the normalized execution time is comparable, or even superior, to the more optimized handwritten state-of-the-art solutions, allowing real-time execution, while meeting the UC requirement in term of acceleration.

## V. CONCLUSION

This work provides additional support for the benefits of using heterogeneous platforms that integrate general-purpose processing with PL to address the needs of modern industrial motion control systems. Specifically, the main goal was to showcase how enhancing system performance in terms of die-cutting machine throughput and die identification capability, as specified by the H2020 project IMOCO4.E, can be achieved by accelerating the VA block. This improvement is made possible through the use of a coprocessing unit, which allows for the integration of a comprehensive vision-based control system on the same computing platform. Performance results indicate that the proposed SoM-based solution meets the timing requirements of the UC. This has been achieved thanks to two DSEs focused on i) assessing accuracy of the VA IPs through a dataset of real input images and ii) improving precision error of the closed-loop system providing the proper acceleration of the VA. In addition, to encourage SW developers in the industrial sector to consider this implementation approach, two strategies were employed. The strategy involves using the AMD Kria KV260 target device, which is designed for advanced vision applications and eliminates the need for extensive HW design expertise. Although it is a “nonproduction” version not intended for final commercial deployment, the AMD K26 family of this SoM also includes a production model with additional connectors, an eMMC nonvolatile memory (allowing the system to boot entirely from the SoM rather than an SD card), and a wider temperature range. This production version supports a complete OS to manage SW execution and facilitate I/O. The development process began with an SW-friendly implementation and progressed to an HW-oriented version using HLS. The final VA IPs achieves the state-of-the-art performance both in terms of execution time and resource usage. The HLS-based design provides an easier path to HW acceleration for developers, allowing them to progressively focus on various types of optimization. In addition, HLS enabled the accuracy-related DSE to be performed more rapidly, since it simplified the implementation of different accelerator configurations.

Further works aimed at improving die position identification could evaluate parametric accelerators based on AI models or exploit the GPU embedded into the SoM for SW-oriented implementations.

## ACKNOWLEDGMENT

Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

## REFERENCES

- [1] G. Van der Veen, J. Stokkermans, N. Mooren, and T. Oomen, “How learning control supports industry 4.0 in semiconductor manufacturing,” 2020.
- [2] Power & Beyond, “Global chip shortage 2023 - updates in june,” 2023. Accessed: Sep. 2024. [Online]. Available: <https://www.power-and-beyond.com/global-chip-shortage-2023-updates-in-june-a-878ccf26f2bb3b7c58ffec27b42b262f/>
- [3] E. A. Lee, “Cyber physical systems: Design challenges,” in *Proc. 11th IEEE Int. Symp. Object Compon.-Oriented Real-Time Distrib. Comput.*, 2008, pp. 363–369, doi: [10.1109/ISORC.2008.25](https://doi.org/10.1109/ISORC.2008.25).
- [4] S. J. Oks, A. Fritzsche, and K. M. Möslin, “Engineering industrial cyber-physical systems: Design challenges,” *Procedia CIRP*, vol. 72, pp. 456–461, 2018, doi: [10.1016/j.procir.2018.03.126](https://doi.org/10.1016/j.procir.2018.03.126).
- [5] S. Mohamed et al., “The imoco4.e reference framework for intelligent motion control systems,” in *Proc. IEEE 28th Int. Conf. Emerg. Technol. Factory Autom.*, 2023, pp. 1–8, doi: [10.1109/ETFA54631.2023.10275410](https://doi.org/10.1109/ETFA54631.2023.10275410).
- [6] R. Girau et al., “Definition and implementation of the cloud infrastructure for the integration of the human digital twin in the social Internet of Things,” *Comput. Netw.*, vol. 251, 2024, Art. no. 110632, doi: [10.1016/j.comnet.2024.110632](https://doi.org/10.1016/j.comnet.2024.110632).
- [7] C. Jugade et al., “Improved positioning precision using a multi-rate multi-sensor in industrial motion control systems,” in *Proc. Eur. Control Conf. (ECC)*, 2023, pp. 1–8, doi: [10.23919/ECC57647.2023.10178138](https://doi.org/10.23919/ECC57647.2023.10178138).
- [8] AMD Xilinx, “Introducing adaptive system-on-modules (SOMs),” 2024. Accessed: Sep. 2024. [Online]. Available: <https://www.xilinx.com/publications/ebooks/introducing-adaptive-system-on-modules.pdf>
- [9] X. Shao and D. Sun, “A FPGA-based motion control IC design,” in *Proc. IEEE Int. Conf. Ind. Technol.*, 2005, pp. 131–136, doi: [10.1109/ICIT.2005.1600623](https://doi.org/10.1109/ICIT.2005.1600623).
- [10] M. A. M. Prado, J. R. Reséndiz, D. C. T. Pérez, C. M. T. Hernández, and G. H. Ruiz, “Motion control with FPGA,” in *Field*, G. Dekoulis, Ed., ch. 3. Rijeka: IntechOpen, 2017.
- [11] J. Wang, M. Li, W. Jiang, Y. Huang, and R. Lin, “A design of FPGA-based neural network PID controller for motion control system,” *Sensors*, vol. 22, no. 3, 2022, doi: [10.3390/s22030889](https://doi.org/10.3390/s22030889).
- [12] C. Rubattu, A. Ledda, F. Ratto, C. Jugade, D. Goswami, and F. Palumbo, “FPGA-based implementation for industrial motion control system,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2024, pp. 141–147, doi: [10.1109/IPDPSW63119.2024.00039](https://doi.org/10.1109/IPDPSW63119.2024.00039).
- [13] Prodrive Technologies B.V., “Motion controllers,” 2024. Accessed: Sep. 2024. [Online]. Available: <https://prodrive-technologies.com/motion/products/motion-controllers/>
- [14] C. Jugade, S. Mohamed, D. Goswami, A. Nelson, G. Van Der Veen, and K. Goossens, “Predictable multi-core implementation of multi-rate sensor fusion for high-precision positioning systems,” in *Proc. Amer. Control Conf. (ACC)*, 2024, pp. 4485–4492, doi: [10.23919/ACC60939.2024.10644161](https://doi.org/10.23919/ACC60939.2024.10644161).
- [15] Intelligent Motion Control under Industry 4.E, “D3.2 perception and instrumentation: Layer 1: Requirements and specifications (final version, V1.0),” 2024. Accessed: Sep. 2024. [Online]. Available: [https://www.imoco4e.eu/app/download/13384334949/D3.2%20Perception%20layer\\_Final\\_1.0.pdf?t=1698915550](https://www.imoco4e.eu/app/download/13384334949/D3.2%20Perception%20layer_Final_1.0.pdf?t=1698915550)
- [16] G. van der Veen, J. Stokkermans, N. Mooren, and T. Oomen, “How learning control supports industry 4.0 in semiconductor manufacturing,” in *Proc. ASPE Spring Topical Meeting 2020: Des. Control Precis. Mechatronic Syst.-Virtual Meeting*. American Society of Precision Engineering (ASPE), 2020, pp. 1–5.
- [17] J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Jun. 1986, doi: [10.1109/TPAMI.1986.4767851](https://doi.org/10.1109/TPAMI.1986.4767851).
- [18] R. O. Duda and P. E. Hart, “Use of the hough transformation to detect lines and curves in pictures,” *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972, doi: [10.1145/361237.361242](https://doi.org/10.1145/361237.361242).

- [19] AMD Xilinx, "Kria KV260 vision AI starter kit," 2024. Accessed: Sep. 2024. [Online]. Available: <https://www.xilinx.com/products/som/kria/kv260-vision-starter-kit.html>
- [20] M. Pelcat, C. Bourrasset, L. Maggiani, and F. Berry, "Design productivity of a high level synthesis compiler versus HDL," in *Proc. Int. Conf. Embedded Comput. Syst.: Architectures, Model. Simul.*, 2016, doi: [10.1109/SAMOS.2016.7818341](https://doi.org/10.1109/SAMOS.2016.7818341).
- [21] M. Levental, A. Khan, R. Chard, K. Yoshii, K. Chard, and I. Foster, "Bragghls: High-level synthesis for low-latency deep neural networks for experimental science," in *Proc. 14th Int. Symp. Highly Efficient Accelerators Reconfigurable Technol.*, 2024, doi: [10.1145/3665283.3665284](https://doi.org/10.1145/3665283.3665284).
- [22] National Instruments, "MATLAB, Simulink, and LabVIEW FPGA: Importing HDL Coder exports into LabVIEW FPGA designs," 2024. Accessed: Sep. 2024. [Online]. Available: <https://www.ni.com/en/support/documentation/supplemental/20/matlab---simulink---and-labview-fpga--importing-hdl-coder--expor.html#section-1952637991>
- [23] A. Xilinx, "Vitis hls," 2024. Accessed: Sep. 2024. [Online]. Available: <https://www.xilinx.com/products/design-tools/vitis/vitis-hls.html>
- [24] Cadence, "Stratus high-level synthesis," 2024. Accessed: Sep. 2024. [Online]. Available: [https://www.cadence.com/en\\_US/home/tools/digital-design-and-signoff/synthesis/stratus-high-level-synthesis.html](https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/stratus-high-level-synthesis.html)
- [25] Siemens, "Catapult high-level synthesis and verification," 2023. Accessed: Sep. 2024. [Online]. Available: <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>
- [26] Microchip, "SmartHLS compiler software," 2023. Accessed: Sep. 2024. [Online]. Available: <https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/smarthls-compiler>
- [27] F. Ratto, T. Fanni, L. Raffo, and C. Sau, "Mutual impact between clock gating and high level synthesis in reconfigurable hardware accelerators," *Electronics*, vol. 10, no. 1, 2021, doi: [10.3390/electronics10010073](https://doi.org/10.3390/electronics10010073).
- [28] C. Sau, L. Fanni, P. Meloni, L. Raffo, and F. Palumbo, "Reconfigurable coprocessors synthesis in the MPEG-RVC domain," in *Proc. Int. Conf. ReConFigurable Comput. FPGAs*, 2015, pp. 1–8, doi: [10.1109/RECONFUG.2015.7393351](https://doi.org/10.1109/RECONFUG.2015.7393351).
- [29] C. Rubattu, F. Palumbo, S. S. Bhattacharyya, and M. Pelcat, "Path-Tracer: Understanding response time of signal processing applications on heterogeneous MPSoCs," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 6, no. 4, 2022, pp. 1–30, doi: [10.1145/3513003](https://doi.org/10.1145/3513003).
- [30] Motion Capture Technologies, "High speed video cameras," 2024. Accessed: Sep. 2024. [Online]. Available: <https://www.mctcameras.com/high-speed-video-cameras/>
- [31] L. Guo and S. Wu, "FPGA implementation of a real-time edge detection system based on an improved canny algorithm," *Appl. Sci.*, vol. 13, no. 2, 2023, pp. 870–886, doi: [10.3390/app13020870](https://doi.org/10.3390/app13020870).
- [32] I. El Hajjoui, S. Mars, Z. Asrih, and A. El Mourabit, "A novel FPGA implementation of hough transform for straight lane detection," *Eng. Sci. Technol., Int. J.*, vol. 23, no. 2, pp. 274–280, 2020, doi: [10.1016/j.jestech.2019.05.008](https://doi.org/10.1016/j.jestech.2019.05.008).
- [33] J. Guan, F. An, X. Zhang, L. Chen, and H. J. Mattausch, "Real-time straight-line detection for XGA-Size videos by hough transform with parallelized voting procedures," *Sensors*, vol. 17, no. 2, 2017, pp. 270–283, doi: [10.3390/s17020270](https://doi.org/10.3390/s17020270).
- [34] W. He and K. Yuan, "An improved canny edge detector and its realization on FPGA," in *Proc. 7th World Congr. Intell. Control Autom.*, 2008, pp. 6561–6564, doi: [10.1109/WCICA.2008.4594570](https://doi.org/10.1109/WCICA.2008.4594570).
- [35] C. Gentsos, C.-L. Sotiropoulou, S. Nikolaidis, and N. Vassiliadis, "Real-time canny edge detection parallel implementation for FPGAs," in *Proc. 17th IEEE Int. Conf. Electron., Circuits Syst.*, 2010, pp. 499–502, doi: [10.1109/ICECS.2010.5724558](https://doi.org/10.1109/ICECS.2010.5724558).
- [36] Q. Xu, S. Varadarajan, C. Chakrabarti, and L. J. Karam, "A distributed canny edge detector: Algorithm and FPGA implementation," *IEEE Trans. Image Process.*, vol. 23, no. 7, pp. 2944–2960, Jul. 2014, doi: [10.1109/TIP.2014.2311656](https://doi.org/10.1109/TIP.2014.2311656).
- [37] D. Sangeetha and P. Deepa, "FPGA implementation of a real-time edge detection system based on an improved canny algorithm," *J. Real-Time Image Process.*, vol. 16, no. 4, pp. 957–970, 2019, doi: [10.1007/s11554-016-0582-2](https://doi.org/10.1007/s11554-016-0582-2).
- [38] X. Lu, L. Song, S. Shen, K. He, S. Yu, and N. Ling, "Parallel hough transform-based straight line detection and its FPGA implementation in embedded vision," *Sensors*, vol. 13, no. 7, pp. 9223–9247, 2013, doi: [10.3390/s130709223](https://doi.org/10.3390/s130709223).
- [39] X. An, E. Shang, J. Song, J. Li, and H. He, "Real-time lane departure warning system based on a single FPGA," *EURASIP J. Image Video Process.*, vol. 38, no. 1, pp. 1–17, 2013, doi: [10.1186/1687-5281-2013-38](https://doi.org/10.1186/1687-5281-2013-38).
- [40] X. Zhou, Y. Ito, and K. Nakano, "An efficient implementation of the gradient-based hough transform using DSP slices and block RAMs on the FPGA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2014, pp. 762–770, doi: [10.1109/IPDPSW.2014.88](https://doi.org/10.1109/IPDPSW.2014.88).



**CLAUDIO RUBATTU** received the M.S. degree in electronic engineering from the University of Cagliari, Cagliari, Italy, in 2015, and the Ph.D. degree in signal, image, vision from INSA Rennes, Rennes, France, in 2020.

He is currently an Assistant Professor with the University of Sassari, Sassari, Italy. He has collaborated with the University of Cagliari and the University of Sassari as a Research Assistant in the context of European projects. He has been consulting as a digital electronic engineer for a private

company working on European Space Agency projects. His research mainly focuses on the management of reconfigurable hardware for embedded systems, CPSs, and edge computing.



**ANTONIO LEDDA** received the B.S. degree in computer science engineering from the University of Sassari, Sassari, Italy, in 2023.

He is currently a Software Developer with Step Ltd., Treviso, Italy. He has been a Research Assistant with the University of Sassari, focusing on HLS-based hardware acceleration of vision algorithms for industrial control systems.



**FRANCESCO RATTO** received the Ph.D. degree in computer engineering from the University of Cagliari, Cagliari, Italy, in 2024.

He is currently an Assistant Professor with the University of Cagliari. He has been a Visiting Student with the TU Dresden, Dresden, Germany, and with the University of Maryland, College Park, MD, USA, and a Postdoctoral Researcher with the University of Sassari, Sassari, Italy. His research is mainly focused on design automation for reconfigurable systems and edge machine learning.

Dr. Ratto serves on the Organization Committee of the CPS Summer School and on the Artifact Evaluation Committee of RAW.



**CHAITANYA JUGADE** received the Master of Technology (M.Tech.) degree in process instrumentation from the College of Engineering Pune, Pune, India, in 2020, and the Bachelor of Technology (B.Tech.) degree in instrumentation and control engineering from the Government College of Engineering Amravati, Amravati, India, in 2015. He is currently working toward the Doctoral degree in electrical engineering with the Electronic Systems Group, Department of Electrical Engineering, Eindhoven University of Technology (TU/e), Eindhoven, The Netherlands.

His research is centered on embedded control systems for high-precision industrial motion control applications. His research interests are platform-aware analysis, codesign, and implementation of feedback control systems on embedded platforms.



**DIP GOSWAMI** (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the National University of Singapore (NUS), Singapore, in 2009.

From 2010 to 2012, he was an Alexander von Humboldt Postdoctoral Fellow with TU Munich, Munich, Germany. He is currently an Associate Professor with the Electronic Systems Group, Department of Electrical Engineering, Eindhoven University of Technology (TU/e), Eindhoven, The Netherlands. His research interests include various

design aspects of embedded control systems in resource-constrained domains, such as automotive and robotics.



**FRANCESCA PALUMBO** (Senior Member, IEEE) received the Ph.D. degree in electronic and computer engineering from the University of Cagliari, Cagliari, Italy, in 2010.

She is currently an Associate Professor with the University of Cagliari, Cagliari, Italy. Her main research interests include embedded, cyber-physical, and reconfigurable systems, as well as code generation tools and design automation strategies for advanced reconfigurable hardware architectures.

Prof. Palumbo is a member of HiPEAC, has been elevated to the Senior Grade within IEEE, and is a member of ACM. She is a permanent Steering Committee Member of the ACM Conference on Computing Frontiers and will serve as General Chair in 2025. She is also an Associate Editor for IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE EMBEDDED SYSTEMS LETTERS, and the Springer *Journal of Signal Processing Systems*. She also joined the IEEE VLSI Systems Applications Technical Committee (VSA-TC) in 2018. She is currently the Scientific Coordinator of the MYRTUS project on cognitive continuum computing technologies. Since 2017, she has been the Director of the CPS Summer School “Designing Cyber-Physical Systems—From concepts to implementation,” now in its sixth edition.